AI SYSTEMS SCIENCE / AGENT SYSTEMS

32讲 · 零基础理论课程 • 第二部分：必备基础技术

05

Token、上下文窗口与信息容量

Tokens · Context Windows · Information Capacity

从一句话为什么会被切成不同数量的Token，到如何逐项计算上下文预算；进一步理解容量、计算缓存、信息定位与长期记忆能力的根本区别。

零基础 · 数字推演　仅理论 · 不含实验　13张矢量图 · 连续案例 · 10题详解

COURSE ROADMAP

本讲学习地图

从Token和Tokenizer的底层编码，推导上下文预算，再进入有效利用、缓存、长周期信息选择与记忆系统设计。

00	本讲学习目标：把“信息容量”变成能推导的模型
01	1. 一个真实会遇到的科研场景：明明有十万字，模型却“看漏”了
02	2. 第一个基础概念：Token到底是什么?
03	3. Tokenizer怎样把文本变成模型可处理的序列?
04	4. 什么是上下文窗口（Context Window）?
05	5. 为模型建立一本“上下文账本”：输入与输出如何算?
06	6. 为什么历史会随着对话不断增长?
07	7. 模型为什么不能“同时记住所有材料”？认识内部处理过程
08	8. 一个容易混淆的词：KV Cache究竟是不是“记忆”？
09	9. “窗口足够长”为什么仍然会答错?
10	10. 容量只是“入口”：完整信息利用链条是什么?
11	11. 用连续30天案例把五种错误真正“演”出来
12	12. 四种常见的信息组织策略：怎么选择?
13	13. 怎样设计“上下文预算”的原则?
14	14. 如何区分四个指标：容量、找回、利用、成本
15	15. 用最少数学把关系重新连起来
16	16. 十个容易混淆的误区
17	17. 本讲完整知识地图：从Token一路推到长期记忆
18	18. 理论思考题（先自己思考，再看答案）
19	19. 理论思考题参考答案与详细推导
20	20. 一页带走的结论与下一讲预告
21	21. 参考文献与原始研究（附直接入口）

贯穿本讲的动态主线

文本→Token→本轮上下文→容量与位置约束→模型使用→系统反馈与记忆

第05讲 Token、上下文窗口与信息容量

课程：《智能体记忆系统：从零基础到科研理解》（32讲理论课程）
单元：第二部分 · 理解记忆系统必需的基础技术（第1讲）
讲义版本：v1.0 · 2026年10月18日
学习方式：完整理论讲解；无需编程、无需实验，不要求预先掌握数学和深度学习。
衔接：前四讲分别讲了记忆的基本定义、外部记忆的必要性、相关信息容器的差别，以及记忆在Agent架构中的位置。本讲往下一层追问：**模型“一次能看见多少信息”，究竟怎样计算？看见很多信息，为什么还可能答错？**

本讲学习目标：把“信息容量”变成能推导的模型

学完本讲，应当能够：①用自己的话解释Token与Tokenizer的关系；②解释为什么Token不是汉字、不是字节、也不等于“一个词”；③判断模型上下文窗口中通常会有什么；④运用一个简单的预算不等式，计算何时装不下资料；⑤区分长上下文容量、有效信息利用、跨会话持久记忆以及推理阶段KV Cache；⑥结合科研Agent的长周期案例解释为什么应该选择、压缩、检索并维护信息，而不是简单地“塞得越多越好”。

> **最重要的整体模型**：人类提供文字/工具提供结果 → Tokenizer把信息编码成Token序列 → 智能体组织本轮输入 → 模型在有限的上下文预算内处理这些Token → 模型生成输出Token → 控制器用工具与记忆维护下一轮的信息。**“模型能容纳”只是第一道门槛，“能找到”“能判断新旧”“能正确使用”是后面的门槛。**

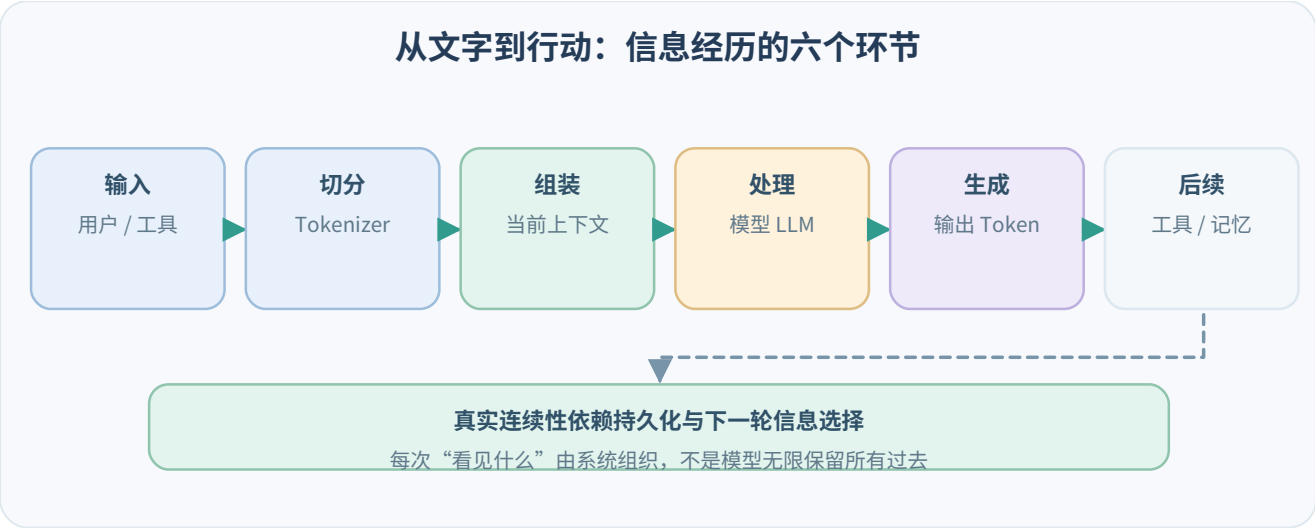


图1 · 从Token编码到行动反馈的动态路径。框图是教学简化，并非各厂商产品的固定实现。

1. 一个真实会遇到的科研场景：明明有十万字，模型却“看漏”了

假设有一个教学虚构的科研助手，持续一个月协助研究者比较不同Agent记忆架构。项目已经产生：数十轮聊天讨论、数篇论文的阅读笔记、对部分实验的评价、两次方向修订，以及一份长篇历史记录。月底研究者提出：**“按照最近确认的研究目标，告诉我下一步应查什么文献，以及为什么。”**

如果智能体只把最近几十条消息提交给语言模型，模型可能不知道第7天的关键决策；如果把所有历史材料毫无筛选地塞进去，又可能发现：输入太长、挤掉输出空间、重复内容过多，甚至旧决策和新决策互相冲突。这些都不是同一种故障。

更隐蔽的情形是：所有信息都确实放进了输入，模型也没有报“超出上下文”，但它仍然沿用了旧计划。**此时失败不一定发生在容量层，而可能发生在信息定位、时间关系理解、证据选择或推理层。**

我们用一条完整因果链理解这一点：

- **容量问题**：想送进去的Token数超过允许上限，有一部分材料无法进入。
- **选择问题**：容量足够，但控制器送进了不重要的信息，没有放入必要证据。
- **利用问题**：重要信息已经进入上下文，但模型没正确找到、比较或使用。
- **时间问题**：旧结论和新决定都存在，模型没有判断哪个仍有效。
- **持久性问题**：本轮完成后，下次新会话没有机制重新取得必要信息。

本讲以“Token与上下文预算”作为入口，最终要解释上述五种问题如何联系，又如何分开诊断。

2. 第一个基础概念：Token到底是什么？

Token（词元）是文本经Tokenizer（分词器/词元化器）处理后，供模型使用的一个基本符号单位。Tokenizer把输入的文字转换成一串离散的Token ID；模型不是按“这一段有多少汉字”直接进行内部运算，而是按编码后的Token序列处理。这里的“分词器”不是传统语文中的“按词语划分”那么简单。

一个Token可能对应一个常见英文单词，也可能只是单词的一部分；可能对应一个汉字、几个汉字，也可能包含标点和空格模式。不同模型的Tokenizer词表不同，同一句话在不同Tokenizer下可能得到不同的Token数。因此，不应记忆“中文一个汉字等于一个Token”“一个英文单词等于一个Token”这样的固定换算公式。

Token ID则是词表中对应条目的编号。比如“研究”可能被一个Tokenizer看作一个单独Token，也可能被另一个Tokenizer拆开；对于系统，最终进入模型的是对应编号以及后续嵌入表示，而非人类直接看见的汉字笔画。编号本身也不是词语含义的数值大小，编号越大不代表词语越重要。

2.1 用一个示意例子认识“切分”

考虑句子：“我们研究智能体记忆。” 以下两种切分只用于解释Token的粒度，不表示某个真实模型的精确切分结果：

- 示意Tokenizer A：`[我们] [研究] [智能体] [记忆] [。]` → 5个Token；

- 示意Tokenizer B：`[我] [们] [研] [究] [智能] [体] [记] [忆] [。]` → 9个Token。

原句完全相同，Token个数却不同。如果上下文上限是按Token计量，就不能直接用“汉字数”代替计算。实际切分必须由**指定模型对应的Tokenizer**得出；这里故意使用两套虚构切分，目的只是展示逻辑。



图2 · 两种虚构切分方式证明Token粒度不固定；此图不是任何真实模型的Tokenizer输出。

2.2 为什么会产生这样的切分？

如果把世界上每个可能的完整句子都做成词表，会需要数量巨大的词条，还无法处理新出现的名称、型号和拼写。如果只用最小字符，又会使序列长度过长。现代语言模型通常采用某种**子词（Subword）或其他离散单元**作为折中方式：高频片段可形成较大的单位，罕见片段仍可拆成更小单位。

Byte Pair Encoding（BPE，字节对编码）是一种有代表性的子词建词表思路。直观过程是：先把文本表示成较小单位，统计哪些相邻片段常常一起出现，然后反复合并高频相邻片段。比如训练语料里“memory”常见，某种Tokenizer就可能把它编码成较大的片段；遇到罕见的技术缩写，可能需要拆成多个片段。不同Tokenizer使用的字节处理、预分词规则、合并规则和特殊Token都可能不同，不能用一个玩具例子概括所有真实实现。[1]

2.3 为什么这对智能体记忆系统重要？

因为**存储信息量的预算，不等于肉眼观察到的文本长度**。一条包含大量项目名、版本号、路径、表格符号和标点的工作日志，看起来不长，却可能占用较多Token。反之，某些高频短语可能编码得较紧凑。不同语言和内容类型的Token效率也可能不同。设计记忆摘要时，不应只规定“保留500个字”，还要考虑真实Tokenizer与最终上下文的消耗。

记忆系统里还可能保存时间、来源、权限、证据编号等元数据。这些信息帮助纠错，但如果全部以自然语言放入模型输入，也会占用Token。**记忆条数≠Token数，Token数也≠有用事实数量。**

3. Tokenizer怎样把文本变成模型可处理的序列?

我们先不研究神经网络，只看数据变换顺序：

第1步：收到原始文本。 例如系统接到“比较A-MEM和MemGPT的记忆组织方式”。

第2步：文本规范化与切分。 按具体Tokenizer规则识别片段；有些采用字节级方案，有些使用其他预分词和词表组织方式。

第3步：查词表映射为ID。 每个Token片段对应词表中的一个或多个ID。常用表示法可写作 $x = (t_1, t_2, ..., t_n)$；这里n是Token数。

第4步：模型把ID转换为向量表示。 这一步一般叫Embedding（嵌入）；数字编号并非直接代表语义，相应的可学习向量和模型内部计算承担后续表示与推理。

第5步：模型生成新的Token ID，再解码为文字。 输出也不是凭空跳出一句完整中文，而是经Token生成与解码得到可读文本。对多数自回归语言模型，生成有逐步向后展开的性质。



图3 · 处理链条的简化展示。Token ID的数值大小本身不是语义强弱。

3.1 为什么输出也需要占据预算?

有些初学者会把“上下文窗口”理解为“我能上传多少文档”，却忽略模型还需要**生成答案**。在许多模型或接口配置中，输入和生成序列受到总上下文长度限制；也有产品把输入限制、最大输出长度分别规定。具体边界取决于模型和服务端，不应把某一家服务的规则说成所有模型的共同规则。

例如一项任务希望模型先阅读长文，再回答2000 Token左右的分析。如果输入已经占满某个总长度窗口，就需要压缩输入、缩短期望输出或换用支持更大容量的配置，而不能要求模型同时无代价地保存全部输入并输出超长答案。

补充边界：除普通文本Token，聊天角色标记、结构化工具调用、多模态编码等也可能产生额外计量；图片、音频等不是简单按汉字数换算。真实产品可能采用不同的计量口径，因此本讲中的数字均用于概念演算，不能代替厂商实时计费规则。

4. 什么是上下文窗口（Context Window）？

上下文窗口可以先理解成：模型在一次处理过程中能够纳入的Token序列范围。对于典型的对话式LLM，请求方会准备一个输入序列，模型依据这些已提供Token形成下一步输出。**窗口是一种“本次可处理的序列容量限制”，不是持久记忆仓库。**

日常类比是：你的书桌能同时铺开多少页材料。书架可以有几百本书，但书桌上一次不可能完整摊开所有书；模型的上下文像书桌，外部记忆或知识库存储像书架。这个比喻有帮助，但要注意：模型不是人，注意力机制也不是肉眼逐字阅读。

4.1 上下文窗口里一般装什么？

在一个典型的长期科研Agent中，当前调用的输入可能包含：**系统/开发者指令**、**本次用户任务**、**近期对话**、**控制器提供的任务状态**、**检索出来的长期记忆**、**读取到的论文片段**、**工具返回的最新观察**。这些不一定全部存在，也不必按固定顺序排列。控制器会根据系统架构决定组装方式。

例如第20天，研究者问：“上周我们否定了哪个备选方案？”模型若能正确回答，通常是因为相关历史事实已被放入当前上下文，或已通过某种内建/外部机制成为本轮可利用的信息。**不能仅凭模型答对一句话，就倒推它一定拥有某种特定的外部记忆模块。**



图4 · 信息在本轮“被组装成输入”，并不因为存在于数据库或聊天页面就自动出现在上下文中。

4.2 一轮、一个会话与跨会话，不能混为一谈

一轮模型调用是“给模型输入→得到输出”的计算行为；**聊天会话**是应用组织多轮用户/助手消息的交互单位；**长期记忆**则是系统有意保存并能在未来重新利用的信息。三者的生命周期可能不同。

聊天界面上“还能向上翻到上个月的记录”，只说明应用层可能保存了历史文本，不证明上个月所有信息此刻都被送给模型；系统可能只选取最近若干消息、发送摘要，或通过检索动态找回部分内容。对用户而言，**“可查看的历史”与“模型当下实际可见的输入”**并不等价。

4.3 大窗口不是无限信息

即使某个模型允许非常长的上下文，它仍受实际资源、运行成本、注意力机制设计、服务端限制和信息组织质量影响。一个很长的窗口提供了容纳更多Token的可能性，却没有承诺每条事实在任何位置都能被稳定取出和正确使用。为什么会这样，我们将在第8、10节深入解释。

5. 为模型建立一本“上下文账本”：输入与输出如何算?

先引入一个简化预算模型：

> **总预算不等式（适用于总长度受同一窗口约束的教学模型）：**
> `T_指令 + T_用户近期 + T_历史 + T_检索资料 + T_工具结果 + T_输出预留 ≤ W`
> 其中 **W** 是总上下文预算（以Token计），每个T表示对应部分Token数量。这里暂不计未单列的协议额外开销；真实系统应把额外开销计入。若模型另设输入与输出独立限制，还必须分别满足它们。

注意这是一条加法约束，不是复杂数学。它说的仅仅是：一个有限大小的盒子里，放进去的不同类别信息加起来不能超过盒子的容量。**下列16,000 Token窗口是完全虚构的教学配置，并不对应任何指定真实模型。**

5.1 原始方案：为什么装不下?

假设研究者要求AI分析最新论文与长期项目决定。智能体控制器尝试把以下信息一股脑放入模型：

占用项目	Token数	为什么要放?
系统与任务规则	900	告诉模型应遵守的范围与格式
本次提问与近期对话	4,200	理解当前工作和最新要求
较早的完整历史	5,800	希望模型知道以前发生了什么
检索的论文片段	3,500	提供文献依据
工具结果及结构化状态	900	体现最近读取/检查的结果
预留输出空间	2,000	生成答案和解释
合计	**17,300**	**超过16,000的总预算**

手工演算：900 + 4,200 = 5,100；再加5,800得10,900；再加3,500得14,400；再加900得15,300；最后加2,000得17,300。**17,300 − 16,000 = 1,300，超出1,300 Token。**



图5 · 全部数字为教学虚构案例。虚线容量轮廓对应16,000，上方彩色长度为17,300。

如果应用直接拒绝请求，错误发生在“容量约束”；如果系统为了不报错偷偷截断一部分输入，仍需要追问**截掉了哪一段**。若刚好把“第15天已否定方案A”的记录删除，系统虽然通过了容量检查，却引入了信息正确性风险。因此，裁剪策略是智能体系统设计的一部分，不是无意义的工程细节。

5.2 改进方案：不是盲目丢弃，而是有条件整理

假设我们保留最新指令与工具观察，对较早历史进行证据可追溯的摘要或按需检索，把5,800 Token历史减少至2,000，同时为了保证原始论文证据，把相关论文片段由3,500增加到4,000。其余部分维持不变，则总量变为：

`900 + 4,200 + 2,000 + 4,000 + 900 + 2,000 = 14,000 Token`。

在16,000总容量下，还留有**2,000 Token**的余量，用于必要的系统额外开销和容量波动。这说明“删除冗余对话”可以给“真正相关的证据”让出空间。不过，不等式满足只证明装得下，**不证明摘要保留了所有关键事实，也不证明最终答案正确**。

5.3 最重要的推断能力：发现约束变化后该改什么

当总量超出上限，可以改变历史回放长度、论文片段数量、结构化状态的呈现方式、输出预留量，或选择不同容量的模型。选择哪一项不能只看省Token最多，还要看**对任务成功最有价值的信息是什么**。如果当前问题依赖被推翻的旧假设及其时间顺序，删除完整决策链就可能严重伤害答案质量。

系统规则和权限检查也不应因“省Token”而随意移除。更高优先级的有效指令由系统的指令机制保证，检索出来的普通记忆不能越权覆盖系统或用户当前明确决定。

6. 为什么历史会随着对话不断增长?

设想只采用“完整历史回放”策略。第一轮你发100 Token，模型答200 Token；第二轮，为让模型看到第一轮，控制器可能重新提交之前的输入和输出，再加上新问题；第三轮又可能加入前两轮。**聊天界面表现为连续对话，不意味着模型内部永久自带整个对话的无限长状态。**

为理解逐轮增长，考虑下面的玩具例子（忽略协议开销与不同轮输出长度）：

轮次	先前历史Token	新任务Token	本次输入合计
第1轮	0	500	500
第2轮	700	300	1,000
第3轮	1,250	250	1,500
第4轮	1,700	400	2,100

这里的“先前历史”已经包含系统选择保留的既往用户/助手内容，数字纯属演示，**不是四轮真实API账单**。趋势很清楚：如果旧内容每轮都保留、还不断追加新消息，输入长度通常增长。实际API计费可以受缓存、供应商规则和请求形态影响，不能简单把上表每一格都视作完整的收费Token。

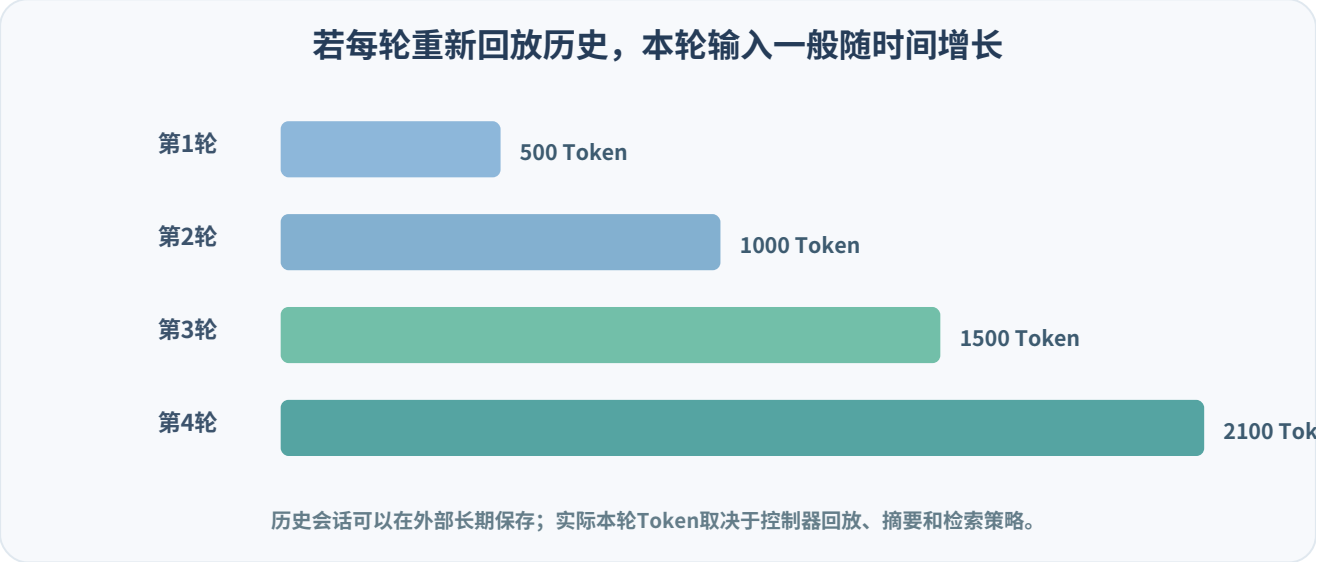


图6 · 纯示意数字，不代表某一API每轮实际计费；缓存和其他优化可能改变具体成本。

6.1 滑动窗口是怎样工作的?

一种朴素方法是“只保留最近N轮”或“只保留最新N个Token”。当新增消息超过限制，最早的部分被移出当前输入。它类似一条移动的取景框：画面永远只看见当前位置附近的片段。

这种方法的优点是控制简单，容量可预测；缺点是时间久了，早期但至今仍有效的用户决定、关键失败经验和任务约束可能被丢掉。**信息离现在很久，不代表它已经失效；信息离现在很近，也不代表它一定重要。**

6.2 滚动摘要为什么也不是万能?

另一种方法把旧历史概括成较短摘要，并把新对话持续纳入摘要。例如原来十天有100条消息，归纳成“本项目当前目标、已完成工作、未决问题”。这样可以控制Token，但也会造成**压缩损失**：弱化来源、时间、否定关系，甚至把“打算做”误写成“已经做”。

如果第2天的摘要出现错误，第10天又根据该摘要重写，第20天继续压缩，错误可能被反复继承。这个现象说明：**压缩不仅减少数量，还改变未来模型能获得的信息结构。** 因此重要决策与证据通常需要可追溯的原始记录或结构化状态，不宜只留下无法核实的一段自然语言总结。

7. 模型为什么不能“同时记住所有材料”？认识内部处理过程

这部分只建立必要的直观模型，不要求你记住矩阵计算。Transformer是一类广泛使用的模型架构，核心部件之一是**注意力机制（Attention）**。简化地说，一个Token的内部表示可以基于其他可见Token的信息进行更新，模型借此综合句子前后的相关内容。[2]

把一句话看成一个班级的同学：每个人要判断自己理解任务时应该参考谁的笔记。注意力机制相当于为某些“参考关系”计算相关权重。这个比喻只能辅助理解，并不代表模型拥有人的主动注意与意识。

经典全连接自注意力（Full Self-Attention）中，长度为n的序列，注意力分数包含大量Token两两关系。粗略数量级可写成 $n \times n = n^2$：4个Token对应16个位置对，8个Token对应64个位置对。这里说的是**典型稠密注意力的关系数量量级**，不是“任何模型推理费用严格等于n^2”。现代系统存在FlashAttention、稀疏/窗口注意力、分页KV管理及各种混合架构，实际显存与运行时间取决于实现细节。[2][3]

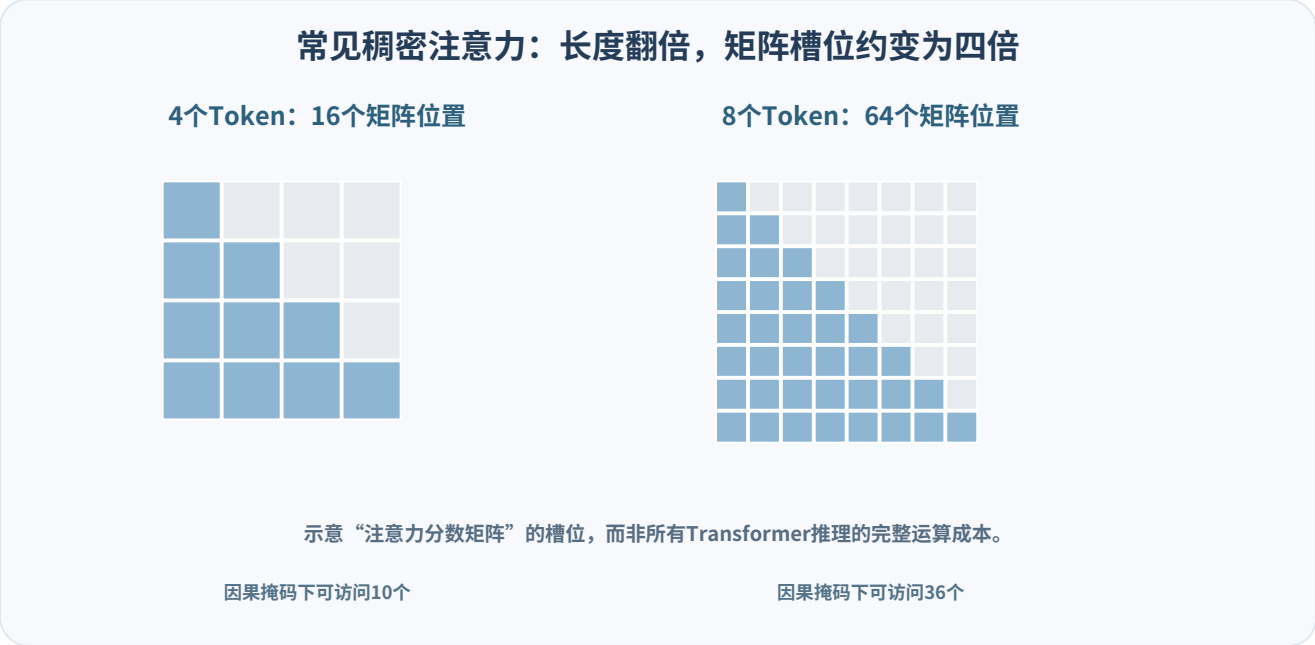


图7 · 典型n×n稠密分数矩阵示意；灰色是未来位置掩码，彩色是可见槽位。真实工程有多种优化。

7.1 为什么“4变8”不是成本简单翻倍?

在上面的稠密注意力玩具模型中，Token数从4变8翻倍，两两关系从16变64，变成4倍。对于更长的序列，这种增长会让计算和数据管理成为重要的工程问题。不过，模型总计算还包含前馈网络、Embedding等其他部分；推理中的预填充与逐Token解码阶段也不同，不能以一个 n^2 公式预测所有平台的收费或速度。

7.2 FlashAttention解决了什么？没有解决什么?

FlashAttention提出一种更重视GPU不同内存层级之间数据搬运的精确注意力算法，通过分块等方法减少不必要的数据读写，从而降低长序列处理的内存与速度开销。[3]

但需要区分：**让处理长输入更高效**，与**让模型在所有长文本位置可靠找到正确事实**，不是同一件事。硬件利用率提升不会自动修正旧结论与新结论的逻辑冲突，更不会替我们决定该把哪段历史保存成长期记忆。

7.3 模型不一定每次把所有历史原样重新计算

工程实现可能采用前缀复用、KV Cache等策略降低重复计算开销，或者只提供历史摘要与检索结果。它们是执行优化和信息选择方法，不能混同于模型“永远记得每一次聊天”。另外，不同模型可能使用不同的注意力形式，某些非典型架构不遵循本节的简化图景。因此这部分教学模型的适用对象是**常见的Transformer语言模型及其变体**。

8. 一个容易混淆的词：KV Cache究竟是不是“记忆”？

KV Cache（Key-Value Cache，键值缓存）是Transformer自回归生成时常用的推理优化机制。其核心思想是：为了生成下一个Token，模型通常需要考虑此前上下文；对已经处理过的前缀，可以缓存若干注意力中间表示（Key和Value），后续生成复用它们，而不必把前面所有中间计算每次完全重算。[7]

这类似做一道长数学题时，把中间算过的结果暂时放在草稿纸上以避免反复计算。请注意：**草稿纸是计算缓存，不是长期档案库。** KV Cache主要服务正在进行或被特定系统复用的推理过程，它通常不以“用户在第7天否定了某科研假设”这样的显式事实形式组织，也不天然提供按主题检索、版本控制、证据来源、时间更新、跨任务权限管理等记忆功能。



图8 · 缓存与记忆按功能区分。缓存能否跨请求复用，取决于具体实现。

8.1 三者怎么区分?

名称	保存的是什么	主要目标	能否直接代替长期Agent记忆?
模型参数	训练得到的权重	模型本身的通用处理能力	不能把个人每次新对话自动写进去
KV Cache	推理中的K/V中间表示	避免重复计算、加速生成	不天然提供长期事实管理、更新与检索
Agent长期记忆	经系统保存的历史信息、状态、经验等	为未来任务提供可用信息	本身也需要检索、选择与正确使用

8.2 KV Cache能否跨请求复用?

可以存在特定实现中的复用，例如多轮对话或共同前缀的缓存。但是“有缓存复用功能”不意味着任何时候、跨任何设备、跨任何新会话都自动保留。缓存复用还依赖模型权重、位置、前缀是否一致，以及服务端的缓存生命周期和隔离机制。它主要优化计算；即使缓存命中，本次答案也可能忽略中间的关键历史事实。

8.3 Cache Memory和Agent Memory是不是一回事?

两者都可能被译成“记忆”，但所解决的系统问题不一样。计算机里的Cache更偏向**快速保留近期计算/数据表示**，而Agent Memory强调**跨时间维护语义信息供未来决策使用**。实际系统可以同时具有两者，却不必把它们融合成同一种对象。

9. “窗口足够长”为什么仍然会答错?

假设一本研究日志有100页，所有页面都进入了模型允许的上下文。相关决定写在第49页。模型能处理这100页，只表示第49页的字节/Token参与了本轮输入表示；是否正确找出该决定，还受任务类型、干扰信息、提示设计、训练和注意力行为等因素影响。

Liu等人在《Lost in the Middle》（TACL 2024）中研究多文档问答和键值检索，发现他们所测模型在一些设置下对**相关信息所处位置**敏感：重要信息位于输入开头或结尾时表现较好，位于中间时性能可能明显下降。[4] 研究图中呈现的典型“U形”是**所研究实验条件下的现象**，并不意味着所有当前模型、所有任务都一定呈U形。

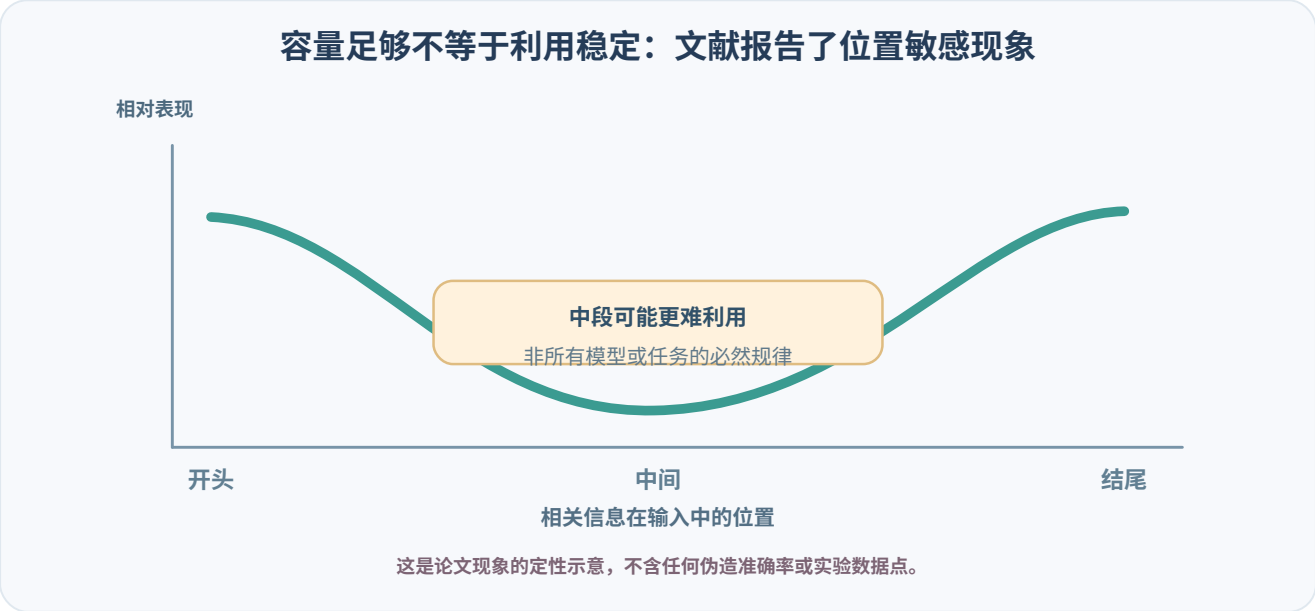


图9 · Liu等人（TACL 2024）报告特定实验下信息位置敏感；曲线仅为教学概念图，无量化数据。

9.1 这里的因果逻辑是什么?

第一层，模型必须**接收到**相关Token；第二层，它要能**定位与识别**相关事实；第三层，它要把事实与问题、时间和来源**正确连接**；第四层，它要据此**形成合理答案**。第一层只关乎容量，第二至第四层涉及信息利用能力。把每层分开，你就不会把所有错误简单解释成“上下文太小”。

9.2 为什么“在长文本里找一根针”并不代表完整记忆能力?

“Needle in a Haystack（大海捞针）”形式的测试，常常把一条易识别的目标事实藏在很长的干扰文本中，询问模型能否找回来。但复杂记忆任务可能要求：找两条相隔很远的历史记录、区分新旧版本、比较前后矛盾、推断有效决策，以及在信息不足时拒绝编造。

RULER（2024）把长上下文评价从简单检索扩展到多目标、多跳追踪、信息聚合等任务，显示对不同复杂度和输入长度，模型能力表现并不相同。[5] 因此，模型宣传的“最大长度”、在单针任务的成绩以及作为长期Agent的可靠表现，是三个不同的观察量。

9.3 长上下文一定不如外部记忆吗?

不一定。对短期任务、静态资料或需要全面比较原文上下文的场景，直接提供足够长且组织良好的原文可能很有效，也避免额外检索错漏。对持续数周、状态频繁变化、历史规模远大于窗口、需要选择性回忆的任务，外部记忆通常更有架构意义。

应该问的不是“长上下文或外部记忆谁永远更好”，而是**在相同任务、可比输入、质量、成本和维护责任下，哪种信息策略更合适**。

10. 容量只是“入口”：完整信息利用链条是什么?

把一个长期科研助手的回答过程分成六道门:

1. **保存（Storage）**：第7天的研究决定有没有在系统里持久保留?
2. **检索（Retrieval）**：第30天被问及时，检索器有没有找到那条决定?
3. **选择（Selection）**：多个候选记录中，系统是否把正确且仍有效的那条放进本轮输入?
4. **容量（Capacity）**：最终组装的上下文是否符合模型的容量和输出限制?
5. **理解（Reasoning）**：模型能否正确比较“原建议—最新修改—当下任务”?
6. **行动（Action）**：模型形成的建议有没有被控制器正确执行并验收?

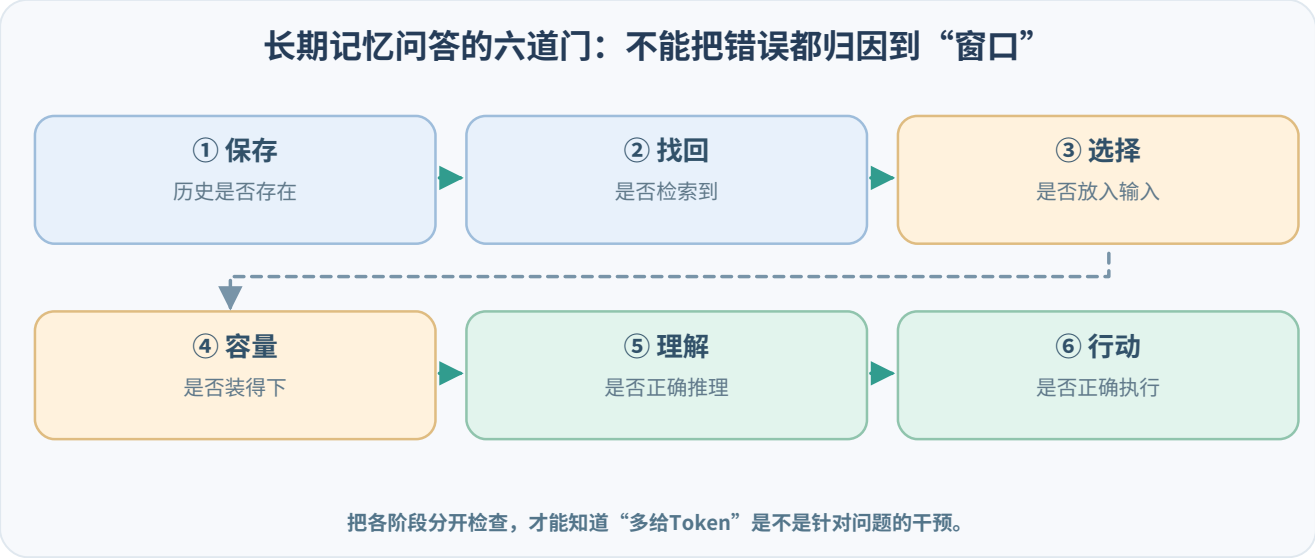


图10 · 保存、检索、选择、容量、理解和行动，是不同的潜在失败层。

有人提出“增加上下文就能解决长时记忆”。这最多在第四道门上提供了余量，也可能间接改善第二、第三道门的策略空间；并没有直接保证其他门成功。真正讨论Agent Memory Systems时，我们要把这六个环节作为不同的设计变量和可能失效点。

10.1 为什么保存量大可能反而更难找?

假设存储里有100,000条事件，只有20条与当前研究决策相关。随着历史增加，检索候选可能有更多语义相近却过时或不适用的信息。哪怕存储成本便宜，系统仍要花资源判断“哪几条适合本次任务”，还要处理互相矛盾的信息。**记忆增长可以增加潜在可用信息，也会增加检索、治理和选择难度。**

10.2 为什么“检索准确”仍可能答错?

检索器可能同时找到了“第7天建议A”和“第15天否决A”。两条都是真实历史。模型若输出“我们现在采用A”，说明它可能把旧建议误当作当前决策。这不是数据没存，也未必是检索没找到，而是**时效性与决策逻辑未正确处理**。

10.3 LongMemEval为什么重要?

LongMemEval（ICLR 2025）面向长期交互助手，明确把评价扩展到信息提取、多会话推理、时间推理、知识更新和必要时拒答等能力，并使用索引、检索、阅读三个阶段梳理系统设计。[6] 对本讲来说，它提醒我们：**长时记忆不是一项纯容量竞赛，而是多环节能力组合。**

11. 用连续30天案例把五种错误真正“演”出来

下面是一份教学虚构的科研项目时间线。假设研究者始终希望在一年左右的研究周期内探究智能体记忆机制，而非急于立项。

时间	历史事实	后续任务可能依赖什么
第1天	项目计划：先系统读综述，不立刻选算法	目标层面的持续约束
第7天	临时认为“全量历史回放”值得优先比较	早期候选，不是终局决定
第15天	阅读证据后，决定不再默认全量回放最优	对第7天想法的修订
第22天	某篇论文笔记指出“长上下文定位不总稳定”	有来源的研究观察，不是我方实验
第30天	要求“按照目前有效的判断提出下一步阅读顺序”	需要第15天最新决定与第22天证据

情况A：容量不足。 控制器只装得下最近两天的对话，第15天的修订没被提交。模型无法凭不存在于当前输入的信息可靠回答过去的精确决定。解决应从保存/检索/预算策略入手。

情况B：历史都进来了，模型答错。 输入包含第7天和第15天两条事实，模型却使用第7天。需要检查时间关系、指令、上下文排列与模型推理，不能只加大窗口。

情况C：摘要丢掉否定词。 旧摘要写成“倾向采用全量回放”，忘记“已取消优先级”。这时错误发生在记忆编码/维护环节，而非窗口或注意力本身。

情况D：检索源不可信。 一条无来源的“方案已获批准”被记忆库收录，相关度较高，被检索放入输入。容量层面一切正常，但记录可信度不足。需要可追溯来源、有效时间与权限边界。

情况E：工具状态过时。 记忆说“论文已下载完成”，磁盘上的文件却已被移动。即使历史记录正确，工具重新核验仍然必要。记忆描述过去，不直接证明当前环境。

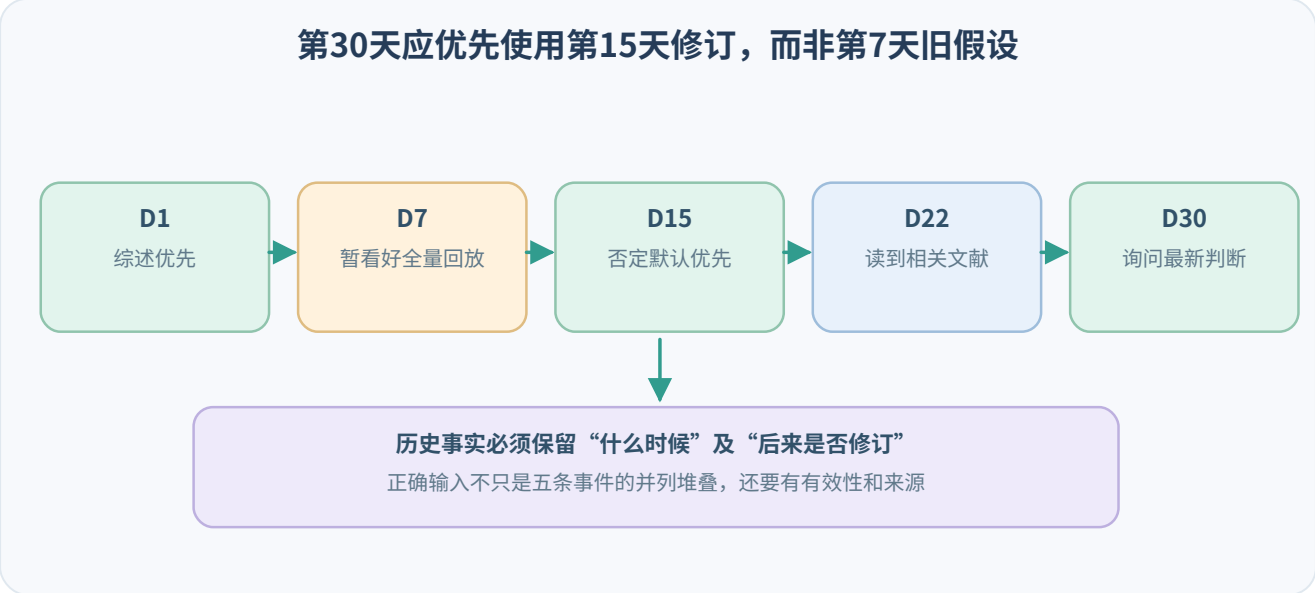


图11 · 虚构五阶段项目：历史记录可同时为真，但当前行动需要理解时间与有效性。

11.1 这五种情况能用同一个“加长上下文”方案解决吗？

不能。A主要是信息没有被提供；B是信息提供以后没正确使用；C是记忆写入就已损失关键语义；D是来源与真伪问题；E是记忆与外部环境不同步。一个Agent Memory研究者需要**给不同故障找不同机制**，否则设计的新系统可能只提高“容纳多少Token”，却对真正的任务失败没有改善。

12. 四种常见的信息组织策略：怎么选择？

把长周期任务的信息带入下一轮，至少有四种常见思路。它们不是必然互斥，也并非从低级到高级的绝对排序。

12.1 方案A：完整历史回放

把过去所有或大量对话内容直接追加到本轮输入。优点是能保留原始细节，不必先定义复杂记忆写入规则；缺点是随历史增长容易超出限制，重复和冲突也会越来越多。适合历史规模仍受控、原始顺序重要、且允许较高上下文开销的任务。

12.2 方案B：滑动窗口

只留最近若干轮或Token。优点是实现简单、成本可控，适合短期连续交互；缺点是无法保障早期重要约束持久可见。若第1天的研究目标到第30天仍有效，简单的“旧消息越早越先丢”就可能错。

12.3 方案C：滚动摘要或压缩

把旧消息压缩成概要，再与新消息结合。优点是能保留大致项目状态；缺点是细节、时间、来源和否定关系可能在反复压缩中丢失。某些系统会把当前摘要与原始日志同时保存，以便必要时回查。

12.4 方案D：长期记忆库+按需检索

将历史内容保存到可持久化系统中，按当前任务检索相关事实、事件与经验，将部分结果放进本轮上下文。优点是检索输入可以小于全部历史规模，并允许独立维护记录；缺点是需要额外的写入、索引、检索、冲突处理与评价机制，容易出现“明明存了却检索不到”。



图12 · 策略示意不是性能排名，选择应取决于任务、质量、延迟和维护成本。

设计取向	完整历史回放	滑动窗口	滚动摘要	外部记忆检索
是否保留原文细节到本轮	较多，受窗口约束	只保留较近部分	主要是概要	取决于检索结果
容量压力	随历史增长	相对可控	相对可控	由检索预算控制
典型缺陷	长文本干扰/成本	遗失早期关键事实	压缩失真	检索漏掉/写入错误
系统维护复杂度	较低起步	较低	中等	通常更高
适合的问题	短周期、要读完整原文	近期连续任务	中期概要状态	长期、选择性回忆

现实系统常用**混合策略**：保留最近对话；维持结构化项目状态；历史记录或文献证据在外部；只把相关部分检索回本轮。策略如何组合，需要针对任务实际验证，而不是仅凭名称判断先进程度。

13. 怎样设计“上下文预算”的原则?

真正重要的问题不是“能否把窗口装满”，而是“在有限Token中，让任务真正需要的信息优先出现”。以下是一套**教学性的上下文组织原则**，不是对任何具体模型/产品的硬性标准：

第一，确保有效的系统约束、授权范围与当下明确任务不被低可信历史覆盖。第二，让当前可靠的工具观察、可核验的状态和最新决定具有明确位置。第三，根据当前问题挑选高度相关的历史记忆，并保留必要的来源和发生时间。第四，保留足够生成输出的余量，避免答案被长度限制截断。第五，在数据变长时优先消除重复、无关和过时内容，而不是一概按时间最早删除。

提示： 你无法仅靠“在Prompt写一句请一定记住”就得到跨会话的可靠长期记忆，除非系统确实有写入、持久化和未来读取机制。同样，检索结果作为上下文材料，不应被当成天然的高优先级系统指令。

13.1 一个简单的效益—成本思维模型

可以把某条候选记忆的选择价值理解成：它与当前任务的**相关程度**、**可信程度**、**时效性**、**有助于行动的程度**，与它所占**Token成本**之间的权衡。不是说把这五项随便相乘就获得真实的科学指标，而是借助它们提出正确问题：

- 这条记录回答了用户的哪个具体问题?
- 它是原始证据、已确认决定，还是模型猜测?
- 它是否已经被更新或否定?
- 少了它会导致什么不同的行动建议?
- 还能用更精确、更短且可回查的形式表达吗?

13.2 为什么要为输出与不可预见开销留余量?

系统在多步工具工作中可能需要新加一段工具回复、状态提示或引用。刚好把窗口填满会降低灵活性；输出长度也是资源的一部分。不过“必须预留10%”之类的固定比例并不是科学定律，应根据模型限制、任务长度和实际服务规则确定。**预算规划是一种可检验的系统设计，而不是经验口诀。**

14. 如何区分四个指标：容量、找回、利用、成本

请把下面四个问题刻意分开。

容量（Capacity）：本轮输入及输出能否在系统约束下被处理?可观察指标有上下文长度、超限率和被裁剪的Token数。

找回（Retrieval）：真正需要的历史事实能否被系统找到并选进当前输入?可以用相关证据的召回率、排序位置等描述，但需要事先知道“正确应该找什么”。

利用（Use / Decision Quality）：模型拿到这些信息后是否产生正确、符合最新约束的答案或行动?需要任务定义、可核对答案、权限与环境成功标准。

成本（Efficiency）：多少输入Token、输出Token、存储/检索开销、计算时间和维护成本换来上述质量？不同系统需要统一比较口径，不能只看单次API账单。

举例：系统甲能装下全部100条历史，系统乙只选出10条关键历史。甲在容量上更大，但如果乙可靠找到最新决定并正确使用，乙的具体任务表现可能更好。相反，如果乙恰好漏掉关键一条，甲也可能获胜。**谁表现更好是需要真实测量的问题，不能从设计名词直接推出。**

15. 用最少数学把关系重新连起来

我们用三个表达式结束这一章的定量部分：

式（1）：容量约束：`T_input + T_reserved_output ⩽ W`。这句负责回答“装不装得下”，不能回答“能不能答对”。

式（2）：当前输入来自信息选择过程：`C_t = Assemble(U_t, S_t, H_recent, Retrieve(M, q_t), O_t)`。这里 `C_t` 是第t轮模型看见的上下文，`U_t` 是用户当前任务，`S_t` 是任务状态，`H_recent` 是近期历史，`M` 是外部记忆，`q_t` 是检索条件，`O_t` 是新的工具观察。“Assemble”表示选择并组织信息的抽象过程；不是某个固定软件函数。

式（3）：行动输出取决于上下文和系统控制：`proposal_t = LLM(C_t)`；随后控制器再检查、执行、观察。即使两种系统的 `W` 相同，如果 `C_t` 的选择方式不同，结果也可能不同。反过来，即使 `C_t` 相同，模型能力、工具和环境也会影响最终成功。

最后得到我们整门课需要记住的系统级结论：**容量是必要资源之一，却不是记忆可靠性的充分条件；长期Agent需要把“保存、检索、组织、推理、执行、更新”放在同一个动态循环里讨论。**

16. 十个容易混淆的误区

1. **“一个汉字就是一个Token。”** 错。Token划分取决于Tokenizer，中文不能固定逐字计数。
2. **“模型支持16K上下文，就能额外无条件输出16K。”** 错。需要看具体总长、输入和输出的各自限制；“16K”本讲只是教学窗口。
3. **“聊天窗口能往上翻，所以模型当前一定看到全部记录。”** 错。应用存储与本轮输入是两个机制。
4. **“上下文越长，回答一定越好。”** 错。信息选择、干扰、时间关系和模型利用能力也影响结果。
5. **“信息进入上下文，就相当于被长期记住。”** 错。跨会话能否回忆取决于持久保存与重新调用。
6. **“KV Cache就是Agent长期记忆。”** 错。缓存推理中间表示与可检索语义档案是不同职责。
7. **“经典注意力是平方量级，所以任何模型的收费都按Token平方增长。”** 错。计算量级和供应商计费体系不是同一个概念。
8. **“把历史全部压缩成一段摘要，重要事实就永远不会丢。”** 错。摘要可能遗漏来源、时间和否定关系。
9. **“模型从历史里检索到一条信息，就说明这条信息现在仍然正确。”** 错。旧记录可能已被推翻，也可能与当前环境不符。

10. **“有更大窗口，就没有必要研究智能体记忆系统。”** 错。窗口容量不能替代长期信息管理、按需检索、更新、权限与系统级评估。

17. 本讲完整知识地图：从Token一路推到长期记忆

用一句连续的话独立复述本讲：**文本会被Tokenizer切成Token，Token形成模型本轮可处理的输入序列；上下文窗口限制当前可容纳的总量，控制器必须决定把哪些近期消息、任务状态、工具结果和历史记忆放进去；即使全部装得下，模型仍可能在长文本里定位失败、误解时间关系或推理错误；因此长期Agent不能只追求更大的窗口，还需要可靠的存储、检索、更新与行动闭环。**



图13 · 期末复述框架：把Token、上下文、信息组织、推理与行动连接成一条动态链。

在后续课程中，第06讲将进一步解释大语言模型怎样利用上下文信息；第07讲讨论Embedding与语义向量；第08讲讨论检索与RAG。请注意这几个主题构成渐进关系：**当前我们先理解窗口里是什么与容量从何而来，之后再学习模型怎样处理与检索信息。**

18. 理论思考题（先自己思考，再看答案）

题1｜概念辨析。 为什么不可以直接说“这篇文章2000个汉字，所以是2000 Token”？指出至少两个原因。

题2｜容量推导。 某教学模型的总窗口预算为12,000 Token。系统规则700、近期对话2,800、文献4,000、工具反馈1,200、历史摘要1,000、输出预留2,000。合计多少？是否超限？如超限，超过多少？

题3｜替代方案。 如果题2超出容量，你能提出两种不同的调整办法吗？说明每一种可能损失的关键信息。

题4｜定位根因。 第1天的重要决定存储在日志中，第20天模型却不知道。能否直接判定“模型注意力不好”？至少给出三个需要进一步检查的环节。

题5｜时效性。 模型在当前输入中同时看到“周一支持方案A”“周四正式否定A”，但周五仍建议A。这个错误为什么不属于典型容量不足？

题6｜缓存辨析。 KV Cache为什么能改善生成效率，却不能直接代替一套长期科研项目记忆系统？

题7｜信息取舍。 两条各占500 Token的候选记忆，一条是当前用户明确确认的研究限制，另一条是旧对话里的无关闲聊。只剩500 Token预算时，应选哪条？为什么不简单依据“更新的日期”排序？

题8｜模型评估。 某模型能把藏在长文本里的编号准确找出来，能否据此得出它能正确处理跨30天的研究方向修订？为什么？

题9｜算术与解释。 16,000 Token预算中原计划总用量17,300。把历史由5,800减到2,000，把论文由3,500增到4,000，其他量不变。新总量是多少？少占用的Token是否代表“答案一定更好”？

题10｜独立重建。 不看知识地图，请用“Tokenizer、上下文、检索、模型、工具、记忆更新”六个词，自主解释智能体如何完成一次跨会话问答。指出至少两个可能失败的环节。

19. 理论思考题参考答案与详细推导

第1题答案。 因为Token不是汉字本身。不同Tokenizer可能合并常见词组或拆开罕见片段，因此同一文本对应Token数量不同；另外标点、空格、英文缩写、版本号等也会影响编码数量。若题目讨论完整API输入，还要考虑角色标记、工具结构等额外内容。正确做法是使用**指定模型的Tokenizer和实际请求格式**计算。

第2题答案。 先算系统与近期：700 + 2,800 = 3,500。加文献4,000得7,500；加工具1,200得8,700；加摘要1,000得9,700；加输出2,000得11,700。**11,700 ⩽ 12,000，所以没有超限，尚余300 Token。** 这道题特意检验是否先算清楚，而不是看见“材料很多”就直觉判断超限；实际系统还可能存在协议开销或独立输出限制，所以300余量不代表一定足够。

第3题答案。 首先应指出题2按简化数字**并未超限**，不存在必须压缩的算术理由。但若实际请求因额外协议Token而超限，一种方法是删去无关旧对话、保留重要决策和来源；风险是误删早期仍有效约束。另一种方法是从4,000Token文献中只选当前问题需要的片段；风险是失去前后文或相关反例。还可在允许的限制内调整输出预留，但不能把输出压缩得无法完成任务。

第4题答案。 不能直接归因注意力。①要看第1天决定是否真的持久保存；②是否在第20天触发检索；③检索器是否找到；④是否被放入当前上下文；⑤上下文是否因容量限制被裁剪；⑥模型收到后是否理解正确。只有查清输入事实，才能进一步判断利用问题。

第5题答案。 两条事实都已进入当前输入，容量至少足以容纳这两条。周五的正确判断应结合时间顺序把周四视为对周一的修订（前提是周四决定来源有效且未有更晚变化）。出错更可能是时效性理解、证据排序或决策逻辑失效，并非“窗口装不下”。

第6题答案。 KV Cache一般保存推理中的注意力K/V中间表示，用于复用先前Token的内部计算，因此可以降低重复计算开销；长期科研记忆则需要保存语义事件、决策、来源、时间、版本及未来可检索组织。缓存本身没有保证提供这些治理机制，并且生命周期与跨任务可复用范围取决于具体系统。

第7题答案。 优先保留**当前已确认且仍有效的研究限制**，因为它直接约束下一步行动，缺失会使建议偏离任务。不能只看日期，因为新近闲聊可能完全无关，而较早但仍生效的决定可能对当前问题至关重要。需要同时判断任务相关性、来源权威性、时效性与约束性质。

第8题答案。 不能。单一目标定位测试主要检验能否从给定材料里找回某条事实，跨30天任务还要保存信息、多会话关联、时间推理、知识更新和必要时拒答。不同能力需要不同的评价设置；RULER与LongMemEval展示了这种问题分解的必要性。[5][6]

第9题答案。 原量17,300；历史减少3,800；论文增加500；新总量为17,300 − 3,800 + 500 = **14,000 Token**。余量16,000 − 14,000 = **2,000 Token**。虽然从容量上可行，但若2,000Token的历史摘要删掉第15天的否定决定，质量可能更差；反之若删掉重复闲聊而补足原始论文证据，质量可能更好。因此质量必须独立检验。

第10题参考答案。 新会话用户提出问题后，控制器依据任务与状态生成检索请求，从长期记忆中找到关联历史；Tokenizer把本轮输入编码成Token，控制器保证上下文容量合适并放入必要信息；模型依据上下文提出回答或调用工具的建议；工具若被调用则返回真实结果，系统核查结果并将有价值的新事实或状态更新到长期记忆。**可能失败处**包括记忆没有保存、检索漏掉、上下文被裁剪、模型误读新旧决定、工具执行失败、写入未经验证的猜测等。答案不要求复述精确术语，关键是能说明数据如何流动和错误在哪发生。

20. 一页带走的结论与下一讲预告

- **Token是模型输入和输出的基本离散单位，不是固定大小的汉字或单词。** 准确数量由指定Tokenizer及请求结构决定。
- **上下文窗口约束模型本次可以处理的序列长度，不是自动跨会话保存所有经历的记忆库。**
- **本轮的信息账本要把有效输入、预留输出及额外开销纳入总量约束。** 计算容量与判断重要性是两个问题。
- **长上下文允许提供更多材料，但不保证模型能稳定定位、比较和正确利用所有信息。**
- **KV Cache用于优化推理计算，不等同于长期语义记忆。**
- **长期智能体必须把保存、检索、选择、容量、理解和行动分开分析。**
- **更大的窗口与更好的记忆系统不是互相排斥的替代关系，而是不同层次的能力。**

下一讲（第06讲）：《大语言模型怎样利用上下文信息？》 我们将从本讲的“Token序列进入模型”继续向下推导：注意力怎样分配、Q/K/V为什么出现、信息位置与生成过程怎样影响长期Agent的行为。仍为零基础理论课，不安排实验。

21. 参考文献与原始研究（附直接入口）

以下资料按“本讲出现的理论节点”选取。教学中的Token切分例子、16K预算和科研项目事件均为**显式虚构的推演材料**，并非来源论文实验数据。

[1] Sennrich, R., Haddow, B., & Birch, A. (2016). **Neural Machine Translation of Rare Words with Subword Units.** ACL 2016, pp. 1715–1725. 关于子词切分和BPE的经典研究。https://aclanthology.org/P16-1162/ DOI: 10.18653/v1/P16-1162

[2] Vaswani, A., et al. (2017). **Attention Is All You Need.** NeurIPS 2017. Transformer架构与注意力机制的重要源头。https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[3] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.** NeurIPS 2022. 高效注意力与GPU数据搬运。https://papers.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html

[4] Liu, N. F., et al. (2024). **Lost in the Middle: How Language Models Use Long Contexts.** Transactions of the Association for Computational Linguistics, 12, 157–173. 长上下文对证据位置的敏感性。https://aclanthology.org/2024.tacl-1.9/ DOI: 10.1162/tacl_a_00638

[5] Hsieh, C.-P., et al. (2024). **RULER: What's the Real Context Size of Your Long-Context Language Models?** arXiv:2404.06654. 长上下文的检索、多跳和聚合评价。https://arxiv.org/abs/2404.06654

[6] Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., & Yu, D. (2025). **LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory.** ICLR 2025. 长期交互记忆评测与索引—检索—阅读结构。https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html

[7] Hugging Face Transformers Documentation. **Caching / KV Cache Explanation.** 官方技术文档，说明自回归生成中K/V缓存的作用、复用与限制。https://huggingface.co/docs/transformers/main/en/cache_explanation

文献阅读顺序建议：初学者先理解[1]与[7]本讲对应的概念，再阅读[4]和[6]的摘要/问题定义；[2][3]涉及较多矩阵运算和GPU系统细节，不需要本周完整掌握，后续第06讲再循序解释。